DBMS - Food Waste Redistribution Platform

1. Conceptual ER Diagram (Chen Notation)

The conceptual model is drawn in traditional Chen notation rather than crow's-foot or table-only notation, so that entity types, relationship types, attributes, keys, cardinality and participation are each shown by a distinct symbol. The model contains 12 entity types (11 strong, 1 weak) and 17 relationship types, all binary.

1.1 Notation used

Symbol	Meaning	Where it appears in this model
Single rectangle	Strong entity type	USER, LOCATION, RESTAURANT, NGO, VOLUNTEER, FOOD_CATEGORY, FOOD_ITEM, DONATION, PICKUP, DELIVERY, AUDIT_LOG
Double rectangle	Weak entity type	FOOD_BATCH — it has no key of its own
Diamond	Relationship type	15 non-identifying relationships
Double diamond	Identifying relationship	HAS_BATCH, which supplies FOOD_BATCH its identity
Oval	Attribute	All 68 attributes
Underlined attribute	Primary key	user_id, location_id, donation_id, …
Dashed-underlined attribute	Partial key	batch_no (unique only inside its FOOD_ITEM)
Double line	Total participation	e.g. every RESTAURANT must be LOCATED_AT a LOCATION
Single line	Partial participation	e.g. a USER need not CONFIRM any DELIVERY
1 / N / M / 0..1	Cardinality ratio	Labelled on every relationship edge

1.2 Entity types

Entity	Type	Primary key	Descriptive attributes
USER	Strong	user_id	full_name, email, password_hash, phone, role, is_active, created_at, updated_at
LOCATION	Strong	location_id	address_line1, address_line2, city, state, postal_code, latitude, longitude
RESTAURANT	Strong	restaurant_id	business_name, contact_name,

			created_at, updated_at
NGO	Strong	ngo_id	organization_name, capacity_kg, is_available, created_at, updated_at
VOLUNTEER	Strong	volunteer_id	availability_status, vehicle_type, created_at, updated_at
FOOD_CATEGORY	Strong	category_id	name, description
FOOD_ITEM	Strong	food_item_id	name, is_vegetarian, description
FOOD_BATCH	Weak	batch_no (partial)	quantity_kg, prepared_at, expires_at, packaging_note
DONATION	Strong	donation_id	status, created_at, accepted_at, cancel_reason
PICKUP	Strong	pickup_id	status, assigned_at, picked_up_at
DELIVERY	Strong	delivery_id	status, delivered_at, confirmed_at
AUDIT_LOG	Strong	audit_id	entity_type, entity_id, action, old_status, new_status, created_at

FOOD_BATCH is weak because a batch number such as "B-01" is meaningful only in the context of the food item it belongs to. Its full conceptual identity is therefore FOOD_ITEM + batch_no, obtained through the identifying relationship HAS_BATCH.

1.3 Relationship types

Relationship	Between	Ratio	Total participation	Meaning
HAS	USER – RESTAURANT	1 : 1	RESTAURANT	A restaurant profile belongs to exactly one login account
HAS	USER – NGO	1 : 1	NGO	An NGO profile belongs to exactly one login account
HAS	USER – VOLUNTEER	1 : 1	VOLUNTEER	A volunteer profile belongs to exactly one login account
CREATES	USER – AUDIT_LOG	1 : N	AUDIT_LOG	Actor who caused an audited change
LOCATED_AT	RESTAURANT – LOCATION	1 : 1	RESTAURANT	Each restaurant has one address
LOCATED_AT	LOCATION – NGO	1 : 1	NGO	Each NGO has one address
OFFERS	RESTAURANT – FOOD_ITEM	1 : N	FOOD_ITEM	Menu items owned by a restaurant
CLASSIFIES	FOOD_CATEGORY – FOOD_ITEM	1 : N	FOOD_ITEM	Each item falls in one category
HAS_BATCH	FOOD_ITEM – FOOD_BATCH	1 : N	FOOD_BATCH	Identifying — gives the weak entity its key
CREATES	FOOD_BATCH – DONATION	1 : 0..1	DONATION	A batch may be donated at most once
ACCEPTS	NGO – FOOD_CATEGORY	M : N	—	Categories an NGO is willing to receive
REQUESTS	NGO – DONATION	N : M	—	Has attributes status, requested_at, reviewed_at

RECEIVED_BY	DONATION – NGO	N : 0..1	—	The one NGO whose request was accepted
HAS	DONATION – PICKUP	1 : 0..1	PICKUP	At most one pickup per donation
ASSIGNED_TO	VOLUNTEER – PICKUP	1 : N	PICKUP	Volunteer carrying out the pickup
RESULTS_IN	PICKUP – DELIVERY	1 : 0..1	DELIVERY	At most one delivery per pickup
CONFIRMS	USER – DELIVERY	1 : N	—	Recipient NGO user confirming receipt

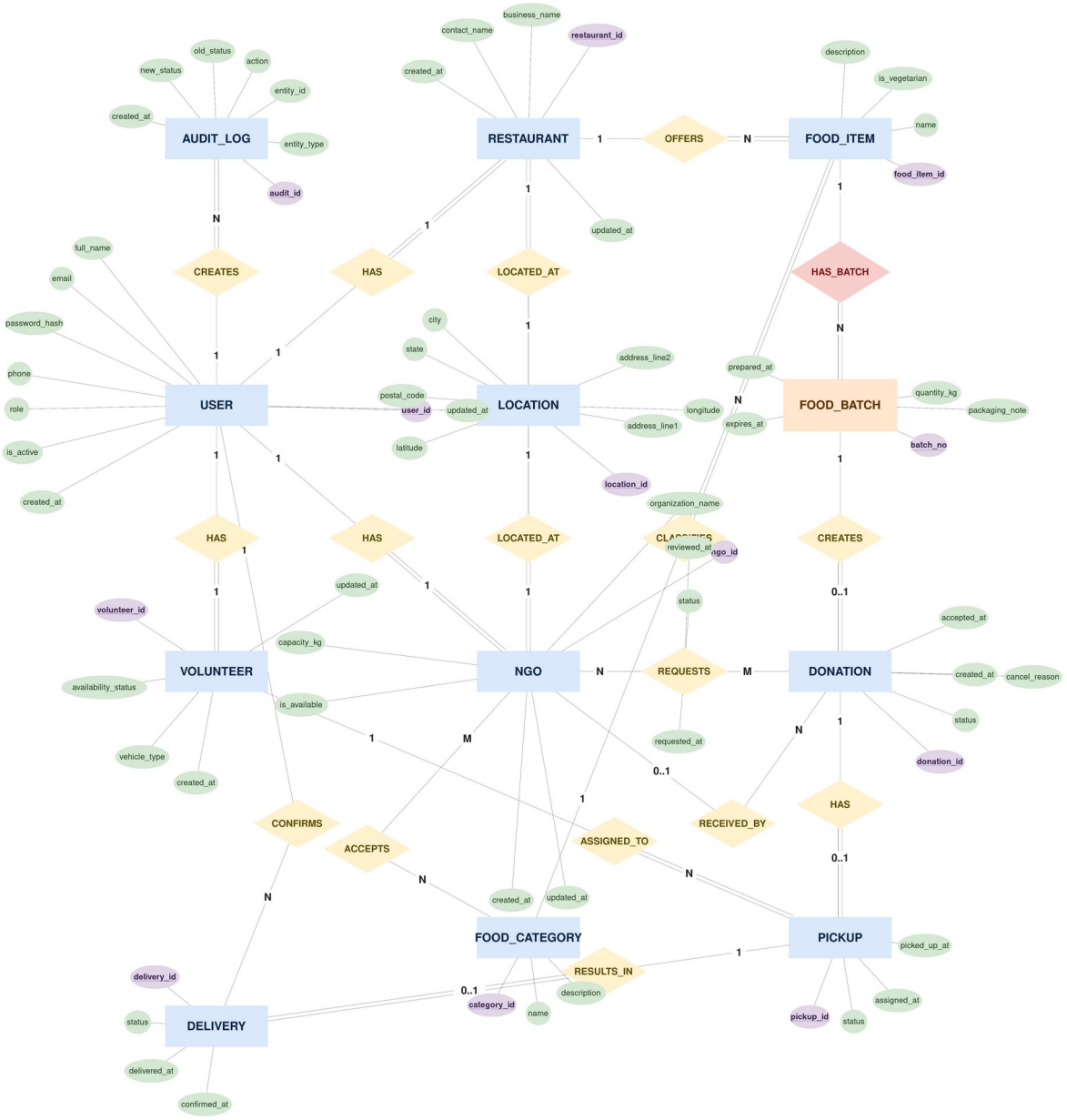
REQUESTS carries its own attributes and is therefore a relationship with attributes, not an entity. ACCEPTS and REQUESTS are the only many-to-many relationships; both become separate relations during mapping (Step 5).

1.4 The diagram

Figure 1 on the following page shows the complete conceptual model: 12 entity types, 17 relationship types and 68 attributes. The page is rotated to landscape so the diagram can be printed at a readable size; the editable source is docs/er-diagram.drawio and a vector copy is docs/er-diagram.svg.

Food Waste Redistribution Platform

Conceptual ER Diagram — Chen Notation



Chen notation

rectangle = strong entity
double rectangle = weak entity (FOOD_BATCH)
diamond = relationship
double diamond = identifying relationship
oval = attribute underlined = primary key
dashed underline = partial key (batch_no)
double line = total participation
1 / N / M / 0..1 = cardinality

2. Relation Mapping (ER → Relational Model)

The conceptual diagram is converted to relations using the standard seven-step ER-to-relational mapping algorithm. Each step is applied below, and the result is the 14-table MySQL schema in Section 3.

2.1 Mapping algorithm applied

Step 1 — Strong entity types

Each of the 11 strong entity types becomes a relation. Its simple attributes become columns and its key attribute becomes the primary key.

> *USER→users, LOCATION→locations, RESTAURANT→restaurants, NGO→ngos, VOLUNTEER→volunteers, FOOD_CATEGORY→food_categories, FOOD_ITEM→food_items, DONATION→donations, PICKUP→pickups, DELIVERY→deliveries, AUDIT_LOG→audit_logs*

Step 2 — Weak entity types

FOOD_BATCH becomes the relation food_batches. The owner's primary key food_item_id is included as a foreign key, and the conceptual primary key is the composite {food_item_id, batch_no}.

> *Implementation note: a surrogate batch_id is kept as the physical primary key so that donations can reference a batch with a single column, while UNIQUE (food_item_id, batch_no) preserves exactly the partial-key semantics of the conceptual model.*

Step 3 — Binary 1:1 relationships

The foreign key is placed in the relation on the side with total participation, and a UNIQUE constraint is added so the 1:1 ratio is actually enforced rather than merely intended.

> *restaurants.user_id, restaurants.location_id, ngos.user_id, ngos.location_id, volunteers.user_id, donations.batch_id, pickups.donation_id, deliveries.pickup_id — every one of them declared UNIQUE.*

Step 4 — Binary 1:N relationships

The primary key of the relation on the 1 side is added as a foreign key to the relation on the N side.

> *food_items.restaurant_id, food_items.category_id, food_batches.food_item_id, pickups.volunteer_id, donations.recipient_ngo_id, deliveries.confirmed_by_user_id, audit_logs.actor_user_id*

Step 5 — Binary M:N relationships

Each many-to-many relationship becomes a new relation whose primary key is the combination of both participating primary keys; relationship attributes become columns of that new relation.

> *ACCEPTS → ngo_food_categories(ngo_id, category_id) with a composite primary key. REQUESTS → donation_requests, which additionally carries status, requested_at and reviewed_at. Because request history must be preserved and reviewed, a surrogate request_id is used with UNIQUE (donation_id, ngo_id) enforcing the M:N key.*

Step 6 — Multivalued attributes

Not applicable. The conceptual model deliberately contains no multivalued attributes.

> *The one repeating fact — the set of categories an NGO accepts — is modelled as the M:N relationship ACCEPTS rather than as a multivalued attribute, which is what keeps the schema free of comma-separated value lists.*

Step 7 — n-ary relationships

Not applicable. All 17 relationship types are binary.

2.2 Mapping table

Every construct in the ER diagram and the relational element it produces:

ER construct	Kind	Maps to	Key / constraint produced
USER	Strong entity	users	PK user_id`; `UNIQUE email`, `UNIQUE phone
LOCATION	Strong entity	locations	PK location_id
RESTAURANT	Strong entity	restaurants	PK restaurant_id`; `UNIQUE business_name
NGO	Strong entity	ngos	PK ngo_id`; `UNIQUE organization_name
VOLUNTEER	Strong entity	volunteers	PK volunteer_id
FOOD_CATEGORY	Strong entity	food_categories	PK category_id`; `UNIQUE name
FOOD_ITEM	Strong entity	food_items	PK food_item_id`; `UNIQUE(restaurant_id, name)
FOOD_BATCH	Weak entity	food_batches	PK batch_id` (surrogate); `UNIQUE(food_item_id, batch_no)
DONATION	Strong entity	donations	PK donation_id
PICKUP	Strong entity	pickups	PK pickup_id
DELIVERY	Strong entity	deliveries	PK delivery_id
AUDIT_LOG	Strong entity	audit_logs	PK audit_id
HAS (USER–RESTAURANT)	1:1	restaurants.user_id	FK → users`, `UNIQUE
HAS (USER–NGO)	1:1	ngos.user_id	FK → users`, `UNIQUE
HAS (USER–VOLUNTEER)	1:1	volunteers.user_id	FK → users`, `UNIQUE
LOCATED_AT (RESTAURANT)	1:1	restaurants.location_id	FK → locations`, `UNIQUE
LOCATED_AT (NGO)	1:1	ngos.location_id	FK → locations`, `UNIQUE
OFFERS	1:N	food_items.restaurant_id	FK → restaurants
CLASSIFIES	1:N	food_items.category_id	FK → food_categories
HAS_BATCH	1:N identifying	food_batches.food_item_id	FK → food_items`, part of the conceptual key
CREATES (BATCH–DONATION)	1:0..1	donations.batch_id	FK → food_batches`, `UNIQUE` (one donation per batch)
ACCEPTS	M:N	ngo_food_categories	PK (ngo_id, category_id)`, both columns FKs
REQUESTS	M:N + attributes	donation_requests	PK request_id`; `UNIQUE(donation_id, ngo_id)
RECEIVED_BY	N:0..1	donations.recipient_ngo_id	FK → ngos`, nullable until acceptance
HAS (DONATION–PICKUP)	1:0..1	pickups.donation_id	FK → donations`, `UNIQUE
ASSIGNED_TO	1:N	pickups.volunteer_id	FK → volunteers
RESULTS_IN	1:0..1	deliveries.pickup_id	FK → pickups`, `UNIQUE
CONFIRMS	1:N	deliveries.confirmed_by_us	FK → users`, nullable, `ON DELETE SET NULL

		er_id	
CREATES (USER–AUDIT_LOG)	1:N	audit_logs.actor_user_id	FK → users`, nullable, `ON DELETE SET NULL

2.3 Resulting relational schema

Primary keys are underlined; foreign keys are shown in italics.

users (user_id, full_name, email, password_hash, phone, role, is_active, created_at, updated_at)

locations (location_id, address_line1, address_line2, city, state, postal_code, latitude, longitude)

restaurants (restaurant_id, *user_id*, *location_id*, business_name, contact_name, created_at, updated_at)

ngos (ngo_id, *user_id*, *location_id*, organization_name, capacity_kg, is_available, created_at, updated_at)

volunteers (volunteer_id, *user_id*, availability_status, vehicle_type, created_at, updated_at)

food_categories (category_id, name, description)

food_items (food_item_id, *restaurant_id*, *category_id*, name, is_vegetarian, description)

food_batches (batch_id, *food_item_id*, batch_no, quantity_kg, prepared_at, expires_at, packaging_note)

donations (donation_id, *batch_id*, *recipient_ngo_id*, status, created_at, accepted_at, cancel_reason)

ngo_food_categories (*ngo_id*, *category_id*)

donation_requests (request_id, *donation_id*, *ngo_id*, status, requested_at, reviewed_at)

pickups (pickup_id, *donation_id*, *volunteer_id*, status, assigned_at, picked_up_at)

deliveries (delivery_id, *pickup_id*, *confirmed_by_user_id*, status, delivered_at, confirmed_at)

audit_logs (audit_id, *actor_user_id*, entity_type, entity_id, action, old_status, new_status, created_at)

2.4 Referential integrity

All 19 foreign keys use ON UPDATE RESTRICT. Deletes are RESTRICT everywhere except the two optional actor references, which are SET NULL so that audit and delivery history survives the removal of a user account.

Child column	References	ON DELETE
restaurants.user_id	users.user_id	RESTRICT
restaurants.location_id	locations.location_id	RESTRICT
ngos.user_id	users.user_id	RESTRICT
ngos.location_id	locations.location_id	RESTRICT
volunteers.user_id	users.user_id	RESTRICT
food_items.restaurant_id	restaurants.restaurant_id	RESTRICT
food_items.category_id	food_categories.category_id	RESTRICT
food_batches.food_item_id	food_items.food_item_id	RESTRICT
donations.batch_id	food_batches.batch_id	RESTRICT
donations.recipient_ngo_id	ngos.ngo_id	RESTRICT
ngo_food_categories.ngo_id	ngos.ngo_id	RESTRICT
ngo_food_categories.category_id	food_categories.category_id	RESTRICT
donation_requests.donation_id	donations.donation_id	RESTRICT
donation_requests.ngo_id	ngos.ngo_id	RESTRICT

pickups.donation_id	donations.donation_id	RESTRICT
pickups.volunteer_id	volunteers.volunteer_id	RESTRICT
deliveries.pickup_id	pickups.pickup_id	RESTRICT
deliveries.confirmed_by_user_id	users.user_id	SET NULL
audit_logs.actor_user_id	users.user_id	SET NULL

The design deliberately avoids a circular dependency: donations does not store an accepted_request_id. The accepted NGO is recorded directly in donations.recipient_ngo_id, so donation_requests depends on donations and never the reverse.

3. DDL Commands

The complete data definition language for the schema. Every statement below is taken verbatim from docs/schema.sql and is executed in this order in MySQL Workbench. All tables use the InnoDB storage engine so that foreign keys, transactions and row-level locking are available, and utf8mb4 so that the full Unicode range is storable.

3.1 Database creation

```
-- Food Waste Redistribution Platform
-- Phase 2A: MySQL 8.0.16+ relational schema
-- Execute this file in MySQL Workbench. All tables use InnoDB and utf8mb4.


CREATE DATABASE IF NOT EXISTS food_redistribution_platform
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;


USE food_redistribution_platform;
```

3.2 Table definitions

users

Shared authentication identity for all four roles. The role ENUM is what makes role-based authorization enforceable in the database rather than only in the UI.

```
CREATE TABLE users (
    user_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(120) NOT NULL,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    phone VARCHAR(25) NULL,
    role ENUM('RESTAURANT', 'NGO', 'VOLUNTEER', 'ADMIN') NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT uq_users_email UNIQUE (email),
    CONSTRAINT uq_users_phone UNIQUE (phone),
    CONSTRAINT chk_users_is_active CHECK (is_active IN (0, 1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

locations

Addresses are normalised into their own relation so that latitude/longitude are stored once and the Haversine distance used by the matching engine has a single source of truth. CHECK constraints bound the coordinates.

```
CREATE TABLE locations (
    location_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    address_line1 VARCHAR(180) NOT NULL,
    address_line2 VARCHAR(180) NULL,
    city VARCHAR(100) NOT NULL,
    state VARCHAR(100) NOT NULL,
    postal_code VARCHAR(20) NOT NULL,
    latitude DECIMAL(9, 6) NOT NULL,
    longitude DECIMAL(9, 6) NOT NULL,
    CONSTRAINT chk_locations_latitude CHECK (latitude BETWEEN -90.000000 AND 90.000000),
    CONSTRAINT chk_locations_longitude CHECK (longitude BETWEEN -180.000000 AND 180.000000)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

restaurants

Three UNIQUE constraints: user_id and location_id enforce the two 1:1 relationships from the ER diagram, business_name prevents duplicate providers.

```
CREATE TABLE restaurants (
  restaurant_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NOT NULL,
  location_id BIGINT UNSIGNED NOT NULL,
  business_name VARCHAR(160) NOT NULL,
  contact_name VARCHAR(120) NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT uq_restaurants_user UNIQUE (user_id),
  CONSTRAINT uq_restaurants_location UNIQUE (location_id),
  CONSTRAINT uq_restaurants_business_name UNIQUE (business_name),
  CONSTRAINT fk_restaurants_user
    FOREIGN KEY (user_id) REFERENCES users (user_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_restaurants_location
    FOREIGN KEY (location_id) REFERENCES locations (location_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

ngos

capacity_kg > 0 is enforced by CHECK. Remaining capacity is never stored — it is derived by query, keeping the relation free of redundant computed data.

```
CREATE TABLE ngos (
  ngo_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NOT NULL,
  location_id BIGINT UNSIGNED NOT NULL,
  organization_name VARCHAR(160) NOT NULL,
  capacity_kg DECIMAL(10, 2) NOT NULL,
  is_available BOOLEAN NOT NULL DEFAULT TRUE,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT uq_ngos_user UNIQUE (user_id),
  CONSTRAINT uq_ngos_location UNIQUE (location_id),
  CONSTRAINT uq_ngos_organization_name UNIQUE (organization_name),
  CONSTRAINT chk_ngos_capacity_kg CHECK (capacity_kg > 0),
  CONSTRAINT chk_ngos_is_available CHECK (is_available IN (0, 1)),
  CONSTRAINT fk_ngos_user
    FOREIGN KEY (user_id) REFERENCES users (user_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_ngos_location
    FOREIGN KEY (location_id) REFERENCES locations (location_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

volunteers

availability_status is an ENUM so only legal states can be written.

```
CREATE TABLE volunteers (
  volunteer_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NOT NULL,
  availability_status ENUM('AVAILABLE', 'UNAVAILABLE') NOT NULL DEFAULT 'AVAILABLE',
  vehicle_type VARCHAR(60) NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT uq_volunteers_user UNIQUE (user_id),
  CONSTRAINT fk_volunteers_user
    FOREIGN KEY (user_id) REFERENCES users (user_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

food_categories

A small reference relation; name is UNIQUE.

```
CREATE TABLE food_categories (
  category_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(80) NOT NULL,
  description VARCHAR(255) NULL,
  CONSTRAINT uq_food_categories_name UNIQUE (name)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

food_items

UNIQUE(restaurant_id, name) stops one restaurant listing the same dish twice while still allowing two restaurants to share a dish name.

```
CREATE TABLE food_items (
    food_item_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    restaurant_id BIGINT UNSIGNED NOT NULL,
    category_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(140) NOT NULL,
    is_vegetarian BOOLEAN NOT NULL DEFAULT TRUE,
    description VARCHAR(255) NULL,
    CONSTRAINT uq_food_items_restaurant_name UNIQUE (restaurant_id, name),
    CONSTRAINT chk_food_items_is_vegetarian CHECK (is_vegetarian IN (0, 1)),
    CONSTRAINT fk_food_items_restaurant
        FOREIGN KEY (restaurant_id) REFERENCES restaurants (restaurant_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_food_items_category
        FOREIGN KEY (category_id) REFERENCES food_categories (category_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    INDEX idx_food_items_restaurant_category (restaurant_id, category_id),
    INDEX idx_food_items_category_id (category_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

food_batches

The weak entity. UNIQUE(food_item_id, batch_no) is the direct translation of the partial key batch_no, and the two CHECK constraints enforce quantity_kg > 0 and expires_at > prepared_at.

```
CREATE TABLE food_batches (
    batch_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    food_item_id BIGINT UNSIGNED NOT NULL,
    batch_no VARCHAR(40) NOT NULL,
    quantity_kg DECIMAL(10, 2) NOT NULL,
    prepared_at DATETIME NOT NULL,
    expires_at DATETIME NOT NULL,
    packaging_note VARCHAR(255) NULL,
    -- FOOD_BATCH is conceptually weak: batch_no is unique only within its FOOD_ITEM.
    CONSTRAINT uq_food_batches_food_item_batch_no UNIQUE (food_item_id, batch_no),
    CONSTRAINT chk_food_batches_quantity_kg CHECK (quantity_kg > 0),
    CONSTRAINT chk_food_batches_expiry_after_preparation CHECK (expires_at > prepared_at),
    CONSTRAINT fk_food_batches_food_item
        FOREIGN KEY (food_item_id) REFERENCES food_items (food_item_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    INDEX idx_food_batches_expires_at (expires_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

donations

UNIQUE(batch_id) enforces one donation per batch. recipient_ngo_id stays NULL until a request is accepted, which is what removes the circular dependency.

```
CREATE TABLE donations (
    donation_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    batch_id BIGINT UNSIGNED NOT NULL,
    recipient_ngo_id BIGINT UNSIGNED NULL,
    status ENUM(
        'AVAILABLE', 'ACCEPTED', 'PICKUP_ASSIGNED', 'PICKED_UP',
        'DELIVERED', 'COMPLETED', 'EXPIRED', 'CANCELLED'
    ) NOT NULL DEFAULT 'AVAILABLE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    accepted_at DATETIME NULL,
    cancel_reason VARCHAR(255) NULL,
    CONSTRAINT uq_donations_batch UNIQUE (batch_id),
    CONSTRAINT fk_donations_batch
        FOREIGN KEY (batch_id) REFERENCES food_batches (batch_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    CONSTRAINT fk_donations_recipient_ngo
        FOREIGN KEY (recipient_ngo_id) REFERENCES ngos (ngo_id)
```

```
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  INDEX idx_donations_status_created_at (status, created_at),
  INDEX idx_donations_recipient_ngo_status (recipient_ngo_id, status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

ngo_food_categories

Pure junction relation from the M:N relationship ACCEPTS; its composite primary key is the mapping-algorithm result.

```
CREATE TABLE ngo_food_categories (
  ngo_id BIGINT UNSIGNED NOT NULL,
  category_id BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY (ngo_id, category_id),
  CONSTRAINT fk_ngo_food_categories_ngo
    FOREIGN KEY (ngo_id) REFERENCES ngos (ngo_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_ngo_food_categories_category
    FOREIGN KEY (category_id) REFERENCES food_categories (category_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  INDEX idx_ngo_food_categories_category_ngo (category_id, ngo_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

donation_requests

Preserves full request history. UNIQUE(donation_id, ngo_id) allows an NGO exactly one request per donation.

```
CREATE TABLE donation_requests (
  request_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  donation_id BIGINT UNSIGNED NOT NULL,
  ngo_id BIGINT UNSIGNED NOT NULL,
  status ENUM('PENDING', 'ACCEPTED', 'REJECTED', 'WITHDRAWN') NOT NULL DEFAULT 'PENDING',
  requested_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reviewed_at DATETIME NULL,
  CONSTRAINT uq_donation_requests_donation_ngo UNIQUE (donation_id, ngo_id),
  CONSTRAINT fk_donation_requests_donation
    FOREIGN KEY (donation_id) REFERENCES donations (donation_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_donation_requests_ngo
    FOREIGN KEY (ngo_id) REFERENCES ngos (ngo_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  INDEX idx_donation_requests_donation_status (donation_id, status),
  INDEX idx_donation_requests_ngo_status (ngo_id, status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

pickups

UNIQUE(donation_id) enforces the 1:0..1 ratio of DONATION–HAS–PICKUP.

```
CREATE TABLE pickups (
  pickup_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  donation_id BIGINT UNSIGNED NOT NULL,
  volunteer_id BIGINT UNSIGNED NOT NULL,
  status ENUM('ASSIGNED', 'PICKED_UP') NOT NULL DEFAULT 'ASSIGNED',
  assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  picked_up_at DATETIME NULL,
  CONSTRAINT uq_pickups_donation UNIQUE (donation_id),
  CONSTRAINT fk_pickups_donation
    FOREIGN KEY (donation_id) REFERENCES donations (donation_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_pickups_volunteer
    FOREIGN KEY (volunteer_id) REFERENCES volunteers (volunteer_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  INDEX idx_pickups_volunteer_status (volunteer_id, status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

deliveries

UNIQUE(pickup_id) enforces one delivery per pickup. confirmed_by_user_id is SET NULL on delete so delivery history outlives the confirming account.

```
CREATE TABLE deliveries (
  delivery_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  pickup_id BIGINT UNSIGNED NOT NULL,
  status ENUM('IN_TRANSIT', 'DELIVERED', 'CONFIRMED') NOT NULL DEFAULT 'IN_TRANSIT',
  delivered_at DATETIME NULL,
  confirmed_at DATETIME NULL,
  confirmed_by_user_id BIGINT UNSIGNED NULL,
  CONSTRAINT uq_deliveries_pickup UNIQUE (pickup_id),
  CONSTRAINT fk_deliveries_pickup
    FOREIGN KEY (pickup_id) REFERENCES pickups (pickup_id)
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT fk_deliveries_confirmed_by_user
    FOREIGN KEY (confirmed_by_user_id) REFERENCES users (user_id)
    ON DELETE SET NULL ON UPDATE RESTRICT,
  INDEX idx_deliveries_status_delivered_at (status, delivered_at),
  INDEX idx_deliveries_confirmed_by_user_id (confirmed_by_user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

audit_logs

Written by the donation lifecycle trigger. actor_user_id is nullable and SET NULL on delete so audit history is never destroyed by removing a user.

```
CREATE TABLE audit_logs (
  audit_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  actor_user_id BIGINT UNSIGNED NULL,
  entity_type VARCHAR(50) NOT NULL,
  entity_id BIGINT UNSIGNED NOT NULL,
  action VARCHAR(80) NOT NULL,
  old_status VARCHAR(30) NULL,
  new_status VARCHAR(30) NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_audit_logs_actor_user
    FOREIGN KEY (actor_user_id) REFERENCES users (user_id)
    ON DELETE SET NULL ON UPDATE RESTRICT,
  INDEX idx_audit_logs_actor_user_id (actor_user_id),
  INDEX idx_audit_logs_entity_created_at (entity_type, entity_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

3.3 Indexes

Indexes are declared inline with each table. They exist to serve real queries, not to inflate a feature count: every one below backs a filter or join used by the application.

Index	Table	Query it supports
UNIQUE (email)	users	Login lookup by email address
(restaurant_id, category_id)	food_items	A restaurant's menu filtered by category
UNIQUE (food_item_id, batch_no)	food_batches	Weak-entity lookup; also serves food_item_id alone
(expires_at)	food_batches	Finding batches close to expiry for urgency scoring
(status, created_at)	donations	Listing AVAILABLE donations newest first
(recipient_ngo_id, status)	donations	An NGO's incoming donations by state
(donation_id, status)	donation_requests	Pending requests for one donation
(ngo_id, status)	donation_requests	One NGO's own request history
(category_id, ngo_id)	ngo_food_categories	Which NGOs accept a given category (matching)
(volunteer_id, status)	pickups	A volunteer's active assignments
(status, delivered_at)	deliveries	Delivery throughput analytics
(entity_type, entity_id, created_at)	audit_logs	Audit trail for one record, chronological

3.4 Constraint summary

Constraint type	Count	Examples
PRIMARY KEY	14	One per relation; ngo_food_categories is composite
FOREIGN KEY	19	All with explicit ON DELETE / ON UPDATE actions
UNIQUE	14	email`, `phone`, `business_name`, `(food_item_id, batch_no)`, `(donation_id, ngo_id)
CHECK	8	quantity_kg > 0`, `capacity_kg > 0`, `expires_at > prepared_at`, latitude/longitude bounds
ENUM	6	role`, `availability_status`, `donations.status`, `donation_requests.status`, `pickups.status`, `deliveries.status
NOT NULL	—	Applied to every attribute that is mandatory in the conceptual model

Constraints that cannot be expressed declaratively — legal donation lifecycle transitions, refusing to accept an expired batch, requiring a recipient NGO before ACCEPTED, and requiring a confirmed delivery before COMPLETED — are enforced by a BEFORE UPDATE trigger on donations, and the atomic acceptance workflow is implemented in the stored procedure sp_accept_donation_request. Both belong to the advanced-DBMS stage of the project and are documented separately.